

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: T.U.P.E.D

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: Aplicaciones de TADs

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

PROBLEMA N° 2 GUERRA DE CARTAS

INTRODUCCIÓN

El informe va a describir y analizar la implementación de la estructura del nuevo programa llamado “Mazo”, desarrollada para completar un juegos de cartas. El trabajo consistió en diseñar un programa que represente un mazo de cartas y permita realizar operaciones fundamentales para saber que cartas se utilizan y como conlleva el juego solo observando de otro punto de vista. Los docentes nos proporcionaron las clases de las cartas y la lógica del juego y la tarea es que en mediante de un tests que sería el juego, nuestro programa funcionará correctamente

DESARROLLO

La clase (DequeEmptyError), está destinada para manejar el error que se produce al intentar extraer una carta de un mazo que está vacío. Lo que evita fallos inesperados y facilita la depuración de errores, haciendo el programa más seguro. Cada uno de los elementos del mazo se representa mediante un nodo doblemente enlazado. De esta manera, se construye una lista doble que permite recorrer el mazo y modificarlo con eficiencia. La clase “Mazo” Esta clase gestiona la baraja de cartas.

Sus principales funciones son:

- Una Inicialización (__init__): Crea un mazo vacío, sin cartas, y define referencias iniciales a su cabeza, cola y tamaño.
- Poner la carta arriba (poner_carta_arriba): Agrega una nueva carta en la parte superior del mazo.
- Poner la carta abajo (poner_carta_abajo): Agrega una carta en la parte inferior del mazo.
- Sacar la carta de arriba (sacar_carta_arriba): Extrae la carta que se encuentra en la parte superior. Si se indica, la carta puede mostrarse como visible. En caso de que no existan cartas, se lanza la excepción personalizada.
- Consultar el tamaño de cartas (__len__): Devuelve el número de cartas contenidas en el mazo.
- Representación en texto (__str__): Permite ver el contenido del mazo en formato de lista, facilitando la comprensión del estado interno de la estructura con la vista de las cartas.

- Verificar si está vacío (`esta_vacio`): Retorna un valor booleano que indica si el mazo hay o no cartas.

`__init__`:

```
def __init__(self):  
    self._cartas = ListaDobleEnlazada()
```

Crea el mazo vacío. Internamente, el mazo no es una lista de Python, sino una `ListaDobleEnlazada`, lo que permite insertar y extraer desde ambos extremos de forma eficiente ($O(1)$).

`poner_carta_arriba` y `poner carta abajo`:

```
def poner_carta_arriba(self, carta):  
    self._cartas.agregar_al_inicio(carta)  
  
def poner_carta_abajo(self, carta):  
    self._cartas.agregar_al_final(carta)
```

- `agregar_al_inicio`: la carta queda en la cima del mazo (posición 0)
- `agregar_al_final`: la carta queda en el fondo del mazo

Esto modela perfectamente un mazo real donde podés poner cartas encima o abajo.

`sacar_carta_arriba`:

```
def sacar_carta_arriba(self, mostrar=False):  
    if self._esta_vacio():  
        raise DequeEmptyError("El mazo está vacío")  
    carta = self._cartas.extraer(0)  # ← usamos extraer directamente  
    if mostrar:  
        carta.visible = True  
    return carta
```

Tiene tres responsabilidades:

- verifica si el mazo está vacío antes de operar
- extrae la carta en el índice 0
- si `motrar= true` marca la carta como visible antes de devolverla

`__esta_vacio:`

```
def __esta_vacio(self):  
    return len(self._cartas) == 0
```

Simplemente consulta si la longitud de la lista es cero. Se usa dentro de `sacar_carta_arriba` como guardia antes de operar.

`__len__` y `__str__`:

```
def __len__(self):  
    return len(self._cartas)  
  
def __str__(self):  
    return "{" + ",".join([str(carta) for carta in self._cartas]) + "}"
```

- `__len__` permite usar `len(mazo)` directamente, como si fuera una lista nativa de Python
- `__str__` permite usar `print(mazo)` y ver algo como `{As♠, 7♥, K♦}`, lo que es útil para depurar o mostrar el estado del juego

CONCLUSIÓN

La implementación del "Mazo" cumplió con lo pedido. Usar una lista doblemente enlazada fue la decisión correcta, ya que permitió manejar las operaciones del mazo de manera eficiente desde ambos extremos. Cada parte del programa funcionó de forma coherente y los tests del juego se ejecutaron sin inconvenientes, validando que la estructura diseñada resuelve el problema correctamente.